[Universidad]
[Departamento]

FUNDAMENTOS DE INFORMÁTICA

Práctica Obligatoria - Curso 2026/2027

ÍNDICE GENERAL

INTRODUCCIÓN 3

1.1. Descripción general de la práctica 3

1.2. Diseño de la práctica: Diagrama de clases 4

ETAPA 1 5

2.1. ETAPA 1.1 - Orientación a Objetos: La clase Anfitrion 5

2.1.1. Objetivos 5

2.1.2. Requisitos previos 5

2.1.3. Implementación de la clase Anfitrion 5

2.2. ETAPA 1.2 Ejecución de Aplicaciones: La clase principal 6

2.2.1. Objetivos 6

2.2.2. Requisitos previos 6

2.2.3. Comprobación de la implementación de la clase principal 6

ETAPA 2 8

3.1. ETAPA 2.1: Uso de la composición 8

3.1.1. Objetivos 8

3.1.2. Requisitos previos 8

3.1.3. Creación de la clase Alojamiento y modificación de la clase Anfitrion 8

3.1.4. Creación de la clase PlataformaAlojamientos (incluye parte Opcional) 9

3.1.5. Comprobación del funcionamiento del mecanismo de composición y de los métodos de listado de anfitriones y alojamientos 11

3.1.6. Comprobación del funcionamiento de las funcionalidades más avanzadas (parte Opcional) 13

ETAPA 3 16

4.1. ETAPA 3.1: Uso de la herencia y polimorfismo 16

4.1.1. Objetivos 16

4.1.2. Requisitos previos 16

4.1.3. Añadiendo alojamientos de diferentes tipos 16

FECHAS Y NORMAS DE ENTREGA 20

5.1. Sesiones presenciales 20

5.2. Normas de entrega 20

5.3. Fechas de entrega 20

5.4. Evaluación de la práctica 20

5.5. Preguntas al equipo docente 21

INTRODUCCIÓN

La práctica se realizará utilizando Java como lenguaje de Programación Orientado a Objetos. El primer concepto que se debe conocer es el de objeto, el ladrillo con el que se construye un programa bajo este paradigma de programación. Los objetos tienen vida cuando nuestro programa está en ejecución, y pueden relacionarse entre sí de diversas maneras, proporcionando al programador las funcionalidades necesarias para resolver una amplia gama de problemas.

La definición de un objeto viene dada por la clase, siendo esta la entidad que en realidad diseñaremos y programaremos en términos de sus campos (o atributos) y métodos. Estos campos y métodos nos permiten definir las características y funcionalidades de la clase. Una vez definida, a partir de la clase podremos crear objetos, que son los que en realidad llevarán a cabo la funcionalidad de nuestro programa en tiempo de ejecución.

Por tanto, podemos ver los objetos como los componentes que nos servirán para diseñar nuestro programa, y las clases como las definiciones de dichos objetos. Una misma clase puede dar lugar a muchos objetos diferentes, cada uno con características diferentes que vendrán dadas por los valores de sus campos.

1.1. Descripción general de la práctica

El propósito de esta práctica es que el alumno desarrolle un sistema informático que simule parte del funcionamiento de una plataforma online de alquiler de alojamientos vacacionales, al estilo de Airbnb[1]. Este proyecto busca familiarizar al estudiante con la creación de aplicaciones informáticas, llevando a la práctica los principios de programación en Java.

En esta práctica se espera que los estudiantes implementen una versión muy simplificada de la plataforma, que solo tendrá como objeto mostrar un catálogo de alojamientos ofertados por un conjunto de anfitriones dados de alta en el sistema.

Se considerarán únicamente dos categorías de alojamientos, urbanos y rurales, y cada alojamiento estará caracterizado por su precio por noche, una descripción y, dependiendo de la categoría, el número de habitaciones, su superficie y planta (urbanos) o la superficie de su parcela y la existencia de piscina (rurales). Los alojamientos tendrán dos estados posibles: disponible u ocupado. Por otro lado, los anfitriones que ofertan los alojamientos en esta plataforma serán identificados por su correo electrónico de contacto y su nombre, y deberán tener asociados los alojamientos que ofertan.

Por tanto, el sistema no contempla ningún tipo de reserva ni de pago; se trata de un sistema que permitirá mostrar un catálogo de alojamientos asociados a anfitriones. El correo electrónico del anfitrión que oferta un alojamiento sería la forma de contacto posterior con la que se realizaría la reserva.

Las funcionalidades básicas que se deberán implementar en esta plataforma serán las siguientes:

- **1. Gestión de anfitriones.** La plataforma ofrecerá la posibilidad de añadir y eliminar anfitriones dentro del sistema. Los anfitriones son aquellas personas que ponen en alquiler algún alojamiento dentro de alguna de las dos categorías consideradas.
- **2. Gestión de alojamientos.** Se deberá permitir que los anfitriones agreguen y retiren alojamientos de su oferta.
- **3. Visualización de alojamientos.** Se podrán mostrar por pantalla los detalles de los diferentes alojamientos ofertados, de modo que si el alojamiento se encuentra en estado disponible se pueda contactar con el anfitrión por medio de correo electrónico para realizar la reserva; como se ha comentado, esta última parte quedaría fuera del alcance de esta práctica.

- **4. Visualización de información de anfitriones.** La plataforma permitirá mostrar por pantalla la información de cualquier anfitrión, incluyendo los alojamientos que ofrece, con sus características y su estado.
- **5. Búsqueda por anfitrión y presentación de alojamientos.** Será posible buscar los alojamientos ofertados por un anfitrión, mostrando aquellos que estén disponibles u ocupados.
- **6. Búsqueda por palabra clave.** Se permitirá la búsqueda de alojamientos que contengan palabras clave específicas en su descripción, y se podrán mostrar en pantalla.

Este ejercicio de programación se centrará únicamente en el back-end, es decir, en la parte del desarrollo encargada de la lógica necesaria para que el sistema funcione, omitiendo la interacción real con una interfaz de usuario y las comunicaciones de red; todo ello será obviado en esta práctica. De este modo, el objetivo es completar la programación de la lógica que subyace a la plataforma planteada, poniendo a prueba la capacidad de los estudiantes para estructurar y organizar el código de una manera que facilite el crecimiento iterativo y la adición de nuevas funcionalidades en futuras versiones.

Para la realización de esta práctica se seguirá un enfoque iterativo, de forma que el desarrollo se dividirá en etapas. En cada etapa se partirá del diseño y desarrollo de la etapa inmediatamente anterior, añadiéndole nuevas funcionalidades y llevando a la práctica los conceptos del temario que se van estudiando progresivamente.

Además, el enunciado de la práctica se acompaña de distintos fragmentos de código, que servirán para validar cada una de las etapas del enunciado, lo que permitirá comprobar si la solución que se está dando en cada momento es correcta o no. Ese código de comprobación estará encapsulado en distintas clases, una por cada etapa, planteando un conjunto de pruebas que permitirán validar la solución propuesta hasta ese momento. Las pruebas consistirán en llamadas a métodos de los objetos creados en cada etapa, por lo que será muy importante respetar y seguir los identificadores que se proponen para los distintos elementos; **deberán respetarse el nombre de las clases, atributos, métodos, tipos de datos, etc.**

[1] https://www.airbnb.es

1.2. Diseño de la práctica: Diagrama de clases

Cuando afrontamos el desarrollo de una aplicación informática es indispensable y necesario realizar una fase previa donde se analicen los requisitos y se diseñen las entidades; en nuestro caso, las clases necesarias para llevar a cabo las funcionalidades requeridas y la relación entre las mismas. Para ello, antes de implementar cada etapa en las que se ha dividido la práctica, se deberá realizar una lectura completa de la etapa a abordar, realizando un diagrama donde se representen las clases necesarias con sus atributos y métodos, así como las relaciones entre las mismas, para posteriormente comenzar la fase de implementación. Así, se deberá realizar un diagrama de clases desde la primera etapa, que luego posteriormente se irá refinando y/o modificando en cada una de las siguientes etapas, siempre como requisito previo a comenzar a implementar el código de cada etapa.

RECOMENDACIÓN: la práctica se realiza a lo largo del cuatrimestre y forma parte del estudio de la asignatura. Es conveniente hacer una primera lectura completa de este documento para tener una visión global de lo que se pide, pudiéndose organizar así apropiadamente el estudio de la asignatura.

El diagrama de clases obtenido en la última etapa debe ser entregado junto al código de la práctica.

ETAPA 1

2.1. ETAPA 1.1 - Orientación a Objetos: La clase Anfitrion

2.1.1. Objetivos

En esta primera etapa de la práctica se van a afianzar los conceptos básicos de la programación orientada a objetos: clase, objeto, atributos y métodos (constructores, métodos que ayudan a la encapsulación de la clase y métodos de visualización).

Veámoslo con un ejemplo que nos permita comenzar esta primera etapa de la práctica. Este año modelaremos una plataforma de alquiler online de alojamientos vacacionales de diferentes categorías, que podrán darse de alta o baja por parte del anfitrión que los oferta, podrá visualizarse su descripción, etc. Por tanto, una entidad básica en este dominio es el anfitrión. Para ello, vamos a empezar diseñando e implementando una clase denominada `Anfitrion`, que nos permitirá encapsular la información asociada a un anfitrión que pone en alquiler alojamientos dentro de nuestra plataforma. Así, cada anfitrión dado de alta en el sistema se modelará mediante un objeto de la clase, con características comunes (atributos o campos), y valores que podrán ser diferentes, representando así cada anfitrión dentro de la plataforma.

2.1.2. Requisitos previos

Para la realización de esta primera etapa se requiere haber estudiado los temas 4, 5 y 6 del temario detallado de la asignatura, correspondientes a los capítulos 1, 2 y 3, así como el apartado 6.13.1 y los apéndices A, B, C, D y F del libro de referencia de la Unidad Didáctica II.

2.1.3. Implementación de la clase Anfitrion

En esta primera etapa realizaremos el modelado inicial de alguno de los componentes principales del sistema. Uno de estos elementos es el anfitrión, que será modelado como una clase a partir de la cual se podrán crear instancias u objetos de la misma. En esta fase del desarrollo de la práctica, cada anfitrión se describe por la siguiente información, que modelaremos como atributos de la clase `Anfitrion`:

- `nombre`, que indicará el nombre del anfitrión que pone un alojamiento en alquiler dentro de la plataforma.
- `email`, el correo electrónico por medio del cual se realizará el contacto en caso de que se quisiera reservar un alojamiento ofertado por el anfitrión, aunque como ya se ha avanzado esta parte queda fuera del alcance de la práctica.

Además, la clase `Anfitrion` deberá permitir consultar y modificar estos campos a través de una serie de métodos creados a tal efecto. Por cada atributo debe haber dos métodos: uno para consultar su valor y otro para modificarlo. Por ejemplo, para el campo `nombre` tendremos los métodos `getNombre()` y `setNombre()` para consultar su valor y modificarlo, respectivamente. Finalmente, la clase `Anfitrion` dispondrá también de un método `print()` que imprima en pantalla toda su información.

Presentamos a continuación en la Figura 2.1 un ejemplo de formato que debería tener la salida en pantalla de este método.

```
Nombre: <nombre>
Email: <email>
```

Figura 2.1: Ejemplo de formato de salida por pantalla mostrando la información de un anfitrión.

RECOMENDACIÓN: practicar con BlueJ creando diferentes objetos de la clase Anfitrion, modificando el valor de sus atributos, mostrándolos por pantalla, etc.

2.2. ETAPA 1.2 Ejecución de Aplicaciones: La clase principal

2.2.1. Objetivos

Es importante comprender que Java y BlueJ son cosas diferentes. Java es un lenguaje de programación, mientras que BlueJ es un entorno de programación que nos permite programar en Java. BlueJ está pensado para el aprendizaje de Java y proporciona distintas herramientas que permiten inspeccionar las partes de un programa, tanto en la parte de diseño (cuando estamos escribiendo el código) como en la de ejecución (cuando el código se ejecuta, se crean los objetos en memoria, etc.).

Una de las herramientas de BlueJ que pueden resultar más útiles es el banco de objetos u *object bench* (ver capítulo 1 del libro), que junto con otras herramientas como el *inspector de objetos*, resulta muy interesante para "jugar" con los objetos en tiempo de ejecución de forma interactiva.

BlueJ nos permite ejecutar nuestras aplicaciones mediante la invocación de los métodos de sus objetos, pero no es esta la única manera de hacerlo. Java, más allá de BlueJ, nos permite ejecutar los programas que creamos de forma independiente. A estas alturas sabemos que cada clase tiene unos métodos y campos propios de los objetos creados a partir de dicha clase, que para poderlos usar necesitamos crear primero un objeto.

RECOMENDACIÓN: Al final de este apartado se debería entender la diferencia entre ejecutar un programa en BlueJ y hacerlo de forma independiente; esto es, ejecutarlo desde la línea de comandos (por ejemplo, desde la consola o el símbolo de sistema en Windows/Linux/MacOS).

2.2.2. Requisitos previos

Además de los expuestos en la sección 2.2.1, deben consultarse capítulos 6.15 y 6.16 del libro de texto, sobre métodos de clase. Además, en el apéndice E puede encontrar más información sobre el método `main()`, en concreto en el apartado E1. Puede consultar también el capítulo 3.10 "Objetos que crean objetos" si necesita más información acerca de la creación de objetos desde el código.

2.2.3. Comprobación de la implementación de la clase principal

Para ejecutar nuestro programa de manera independiente de BlueJ y crear el flujo principal del sistema, crearemos una clase llamada `LanzadorEtapa1` e implementaremos su método `main()`. Esta clase contendrá el código en el que se crean las instancias de objetos necesarias y las llamadas para poder comprobar el funcionamiento del sistema. El ejercicio a realizar en este punto consiste en crear dicha clase, que contenga un método `main()` e implementar el siguiente código[2]:

```
public class LanzadorEtapa1 {
    public static void main(String[] args) {
        Anfitrion anf1 = new Anfitrion("Lucía Mora Vidal","lucia.mora@correo.es");
        Anfitrion anf2 = new Anfitrion("Pablo Sanz Rey","pablosanz@mimail.com");
        Anfitrion anf3 = new Anfitrion("Elena Ruiz Cano","elenaruiz@otromail.com");
        anf1.print();
        anf2.print();
        anf3.print();
    }
}
```

Este código debería crear tres instancias de la clase `Anfitrion`. Posteriormente, el código imprime por pantalla la información de todos los anfitriones creados. Al ejecutar el `main()` de este `LanzadorEtapa1`, el sistema deberá imprimir por pantalla la siguiente información:

```
Nombre: Lucía Mora Vidal
Email: lucia.mora@correo.es
Nombre: Pablo Sanz Rey
Email: pablosanz@mimail.com
Nombre: Elena Ruiz Cano
Email: elenaruiz@otromail.com
```

Se recomienda probar con más objetos y valores de sus atributos. Como ejercicio, debe ejecutarse esta clase `LanzadorEtapa1`, comprobando que compila y ejecuta correctamente, visualizándose los datos de los anfitriones. A continuación, modifique el método `main()` para crear nuevos objetos de la clase `Anfitrion`, asignando los valores que desee para sus atributos y visualícelos de manera análoga. Ejecute el programa desde BlueJ y desde la línea de comandos (consola o símbolo de sistema, según el sistema operativo que utilice).

[2] En el curso virtual, junto con el enunciado de la práctica, puede descargar el código fuente de esta clase LanzadorEtapa1.

NOTA IMPORTANTE: La presente práctica debe poderse ejecutar independientemente de BlueJ de cara a su entrega y corrección.

ETAPA 2

3.1. ETAPA 2.1: Uso de la composición

3.1.1. Objetivos

Una manera muy usual de que los objetos se relacionen en la programación orientada a objetos es mediante el mecanismo de composición. Esta relación se produce cuando un objeto contiene a otros objetos para realizar alguna tarea con ellos. En Java tenemos distintas formas de aplicar este mecanismo; por un lado, podemos tener un objeto como atributo de una clase, pero también podemos emplear estructuras de datos que pueden almacenar conjuntos de objetos, permitiéndonos realizar otro tipo de relaciones de composición. Algunos ejemplos de estas estructuras son: las listas, los conjuntos o los mapas.

En esta segunda etapa de la práctica haremos uso de la composición de la siguiente forma:

- En primer lugar, modificando la definición de la clase `Anfitrion` para incluir un atributo con la lista de alojamientos que quiere poner en alquiler dentro de la plataforma.
- En segundo lugar, la composición se aplicará para integrar una nueva clase que habrá que crear, la clase `PlataformaAlojamientos`, y que será la encargada de gestionar las altas y bajas de cada uno de los anfitriones dentro de la plataforma, así como de asignar alojamientos a anfitriones. Permitirá también acceder a información de alojamientos a través de los anfitriones que los ofertan.

En resumen, deberá crearse una nueva clase `PlataformaAlojamientos` que tendrá como atributo una lista de objetos de la clase `Anfitrion`. También será necesario modificar la clase `Anfitrion` creada en la etapa anterior para incluir como campo una lista de objetos de una nueva clase `Alojamiento` que también deberemos crear.

En esta etapa se aplicarán los principios de encapsulación de la programación orientada a objetos, según se describe en el libro base de la asignatura. Esto implica que los atributos de las clases deben estar definidos como privados, siendo los métodos de la propia clase los únicos que tienen acceso a dichos atributos.

Finalmente, es una buena práctica de programación documentar todo el código que se va escribiendo, no solo con el objetivo de que terceras personas puedan comprender nuestro programa, sino también para que nosotros mismos podamos entender nuestro código de cara a futuras extensiones, mejoras o modificaciones. Para la generación de documentación, Java nos ofrece la herramienta Javadoc, que por medio de determinado tipo de comentarios añadidos en nuestro código fuente, nos permite la generación posterior de toda la documentación de las distintas clases.

3.1.2. Requisitos previos

Para la realización de esta etapa se deben estudiar los temas 7, 8 y 9 del temario detallado de la asignatura. Encontrará información más detallada sobre el mecanismo de composición y la generación de documentación en el libro de texto de la asignatura en los capítulos 4, 5 y 6. La lectura de las secciones de 8.3 a 8.6 (ambas inclusive) sobre buenas prácticas en el diseño de aplicaciones es también recomendable.

3.1.3. Creación de la clase Alojamiento y modificación de la clase Anfitrion

Nuestra aplicación permitirá gestionar la oferta de alojamientos vacacionales que modelaremos mediante una clase `Alojamiento`. Pero hasta ahora, la clase `Anfitrion` no tenía ningún atributo para guardar los alojamientos que ofertaba un anfitrión, de modo que también habrá que modificar esta clase para incluirlo. Ahora queremos que `Anfitrion` tenga como atributo una lista de objetos de una clase `Alojamiento`.

RECOMENDACIÓN: Lea primero los requisitos completos antes de tomar decisiones acerca de cómo desarrollar la solución.

Por tanto, en primer lugar, deberemos crear una clase `Alojamiento` con los siguientes atributos:

- `id`, que permitirá identificar unívocamente un alojamiento dentro del sistema.
- `titulo`, que indicará el texto principal que se pondrá en el anuncio.
- `precioNoche`, que establecerá el precio por noche del alojamiento (tipo `double`).
- `descripcion`, que contendrá el texto que describe las características del alojamiento.
- `estado`, que mostrará el estado actual del alojamiento. Para ello usaremos un tipo enumerado `T_Estado` que habrá que crear con los valores posibles: `DISPONIBLE` u `OCUPADO`.

Aparte de los campos anteriores, habrá que implementar métodos que permitan consultar y modificar la información de todos ellos (`getId()`, `setPrecioNoche()`, etc.). El método `setEstado()` deberá imprimir por pantalla un mensaje informando del cambio, como se muestra más adelante. También deberá poder mostrarse por pantalla la información completa de un alojamiento por medio de un método `print()`. El formato de salida por pantalla será el mostrado en la Figura 3.1. El constructor de la clase recibirá, por este orden, `id`, `titulo`, `precioNoche`, `descripcion` y `estado`, e imprimirá un mensaje de creación.

```
ID: <id>
Titulo: <titulo>
Precio por noche: <precioNoche>
Descripcion: <descripcion>
Estado: <DISPONIBLE u OCUPADO>
```

Figura 3.1: Ejemplo de formato de salida por pantalla mostrando la información de un alojamiento.

Una vez creada la clase `Alojamiento`, se deberá incluir un atributo `alojamientos` en la clase `Anfitrion` para almacenar los alojamientos que oferte un anfitrión. Este atributo deberá ser una lista de objetos de la clase `Alojamiento`. Se deberá permitir añadir y eliminar alojamientos a esta lista, de modo que en el momento en el que se incluya un alojamiento en la lista se fije por defecto su estado como disponible. Además, el constructor de `Anfitrion` imprimirá ahora un mensaje de creación.

El método `print()` de la clase `Anfitrion` deberá ser modificado también para mantener un formato de salida similar al mostrado en la Figura 2.1. La Figura 3.2 muestra un ejemplo del formato que deberá tener ahora la salida por pantalla del método en esta etapa 2, incluyendo también los datos de todos los alojamientos que tiene en oferta el anfitrión.

```
------------
Nombre: <nombre>
Email: <email>
------------
ID: <id>
Titulo: <titulo>
Precio por noche: <precioNoche>
Descripcion: <descripcion>
Estado: <DISPONIBLE u OCUPADO>
ID: <id>
Titulo: <titulo>
...
```

Figura 3.2: Ejemplo de formato de salida por pantalla mostrando la información de un anfitrión en la etapa 2. Se repite el bloque de datos por cada alojamiento que tenga en oferta.

3.1.4. Creación de la clase PlataformaAlojamientos (incluye parte Opcional)

Luego se creará una clase que permita la gestión del sistema, la gestión de anfitriones y sus alojamientos en oferta. Nuestra plataforma de alquiler se gestionará por medio de una clase `PlataformaAlojamientos`, que deberá contar entonces con un atributo `anfitriones` para guardar la lista de anfitriones del sistema. Incluiremos también otro atributo `nombre` para denominar nuestra plataforma de alquiler. Su constructor recibirá el nombre e imprimirá un mensaje de creación.

El comportamiento de esta clase que permite la gestión del sistema debe cubrir las siguientes funcionalidades:

- 1. Se podrá dar de alta un nuevo anfitrión en el sistema con toda la información correspondiente al mismo, es decir, se deberá permitir añadir un objeto de la clase `Anfitrion` al atributo `anfitriones`.
- 2. Se permitirá eliminar anfitriones del sistema a partir de su email.
- 3. Se podrá asociar un alojamiento a un anfitrión dado su email. De modo análogo, se deberá permitir eliminar un alojamiento de la lista de alojamientos de un anfitrión, dados el id del alojamiento y el email del anfitrión.
- 4. Se podrá listar por pantalla el conjunto de alojamientos que se encuentran en oferta en la plataforma, así como el de anfitriones dados de alta.

(Parte Opcional) Esta parte de la práctica correspondiente a las funcionalidades 5, 6 y 7 es opcional. La correcta solución de la misma permitirá al estudiante optar a la máxima calificación de la práctica (Sobresaliente), no permitiendo compensar deficiencias en otras partes obligatorias del enunciado, siendo un notable la máxima calificación a la que se puede optar sin realizar esta parte.

- 5. Se permitirá también imprimir el listado según el estado de los alojamientos, es decir, el conjunto de todos los alojamientos disponibles u ocupados dentro del sistema.
- 6. Se podrá listar por pantalla el conjunto de alojamientos que ofrece un determinado anfitrión, con un determinado estado y a partir de su correo electrónico.
- 7. Por último, se permitirá buscar alojamientos dentro del sistema a partir de una palabra clave que aparezca en su descripción (sin distinguir entre mayúsculas y minúsculas), mostrando por pantalla la información de cada uno de ellos.

Los identificadores que deberá tener la clase `PlataformaAlojamientos` son los siguientes:

Método	**Funcionalidad**
addAnfitrion(Anfitrion)	Da de alta un anfitrión (1)
removeAnfitrion(String email)	Elimina un anfitrión a partir de su email (2)
addAlojamiento2Anfitrion(Alojamiento, String email)	Asocia un alojamiento a un anfitrión (3)
removeAlojamiento2Anfitrion(int id, String email)	Elimina un alojamiento de un anfitrión (3)
printAllAnfitriones()	Imprime todos los anfitriones con sus alojamientos (4)
printAllAlojamientos()	Imprime todos los alojamientos de la plataforma (4)
printAlojamientosByAnfitrion(String email)	Imprime los alojamientos de un anfitrión (4)
printAlojamientosByState(T_Estado)	Imprime los alojamientos con un estado dado (5, opcional)
printAlojamientosByAnfitrion(String email, T_Estado)	Imprime los alojamientos de un anfitrión con un estado dado (6, opcional)
searchAlojamientosByKeyword(String)	Imprime los alojamientos cuya descripción contiene la palabra (7, opcional)

En la creación de esta clase que permite la gestión del sistema es crucial tener en cuenta el concepto de encapsulación. La clase `PlataformaAlojamientos` se encarga de mantener la lista de anfitriones dados de alta dentro del sistema. Adelantamos que en la etapa posterior querremos considerar diferentes categorías de alojamientos con características distintas; en términos de programación orientada a objetos esto implica que la información que vayamos a imprimir por pantalla de cada alojamiento será diferente en un caso u otro.

Llegados a este punto del desarrollo de la práctica, deben documentarse todas las clases creadas hasta el momento. En el capítulo 6, concretamente en la sección 6.11, se describen los pasos a seguir para documentar las clases correctamente.

3.1.5. Comprobación del funcionamiento del mecanismo de composición y de los métodos de listado de anfitriones y alojamientos

Vamos a crear ahora una nueva clase llamada `LanzadorEtapa2_1`, que contenga un método `main()` e implementar en este método el siguiente código[3]:

```
public class LanzadorEtapa2_1 {
    public static void main(String[] args) {
        System.out.println("-----------------------------------");
        System.out.println("Creamos la plataforma de alojamientos");
        System.out.println("-----------------------------------");
        PlataformaAlojamientos gestor = new PlataformaAlojamientos("Vacanza");
        System.out.println("-------------------");
        System.out.println("Creamos anfitriones");
        System.out.println("-------------------");
        Anfitrion anfitrion1 = new Anfitrion("Lucía Mora Vidal","lucia.mora@correo.es");
        Anfitrion anfitrion2 = new Anfitrion("Pablo Sanz Rey","pablosanz@mimail.com");
        Anfitrion anfitrion3 = new Anfitrion("Elena Ruiz Cano","elenaruiz@otromail.com");
        System.out.println("--------------------");
        System.out.println("Añadimos anfitriones");
        System.out.println("--------------------");
        gestor.addAnfitrion(anfitrion1);
        gestor.addAnfitrion(anfitrion2);
        gestor.addAnfitrion(anfitrion3);
        System.out.println("--------------------");
        System.out.println("Creamos alojamientos");
        System.out.println("--------------------");
        Alojamiento alojamiento1 = new Alojamiento(1,"Apartamento céntrico en Madrid",95,
                "Luminoso, junto a la Puerta del Sol. Wifi y aire acondicionado.",T_Estado.DISPONIBLE);
        Alojamiento alojamiento2 = new Alojamiento(2,"Casa rural en la sierra de Gredos",140,
                "Casa de piedra con chimenea y barbacoa. Se admiten mascotas.",T_Estado.DISPONIBLE);
        Alojamiento alojamiento3 = new Alojamiento(3,"Estudio en la playa de Valencia",70,
                "Estudio nuevo a 50 metros de la playa. Cocina equipada.",T_Estado.DISPONIBLE);
        Alojamiento alojamiento4 = new Alojamiento(4,"Loft con vistas en Barcelona",120.5,
                "Loft moderno con vistas a la ciudad, cerca del metro.",T_Estado.DISPONIBLE);
        Alojamiento alojamiento5 = new Alojamiento(5,"Cabaña junto al lago",85,
                "Cabaña de madera junto al lago con barbacoa y zona de pesca.",T_Estado.DISPONIBLE);
        System.out.println("-----------------------------------------");
        System.out.println("Añadimos los alojamientos a los anfitriones");
        System.out.println("-----------------------------------------");
        gestor.addAlojamiento2Anfitrion(alojamiento1,"lucia.mora@correo.es");
        gestor.addAlojamiento2Anfitrion(alojamiento2,"pablosanz@mimail.com");
        gestor.addAlojamiento2Anfitrion(alojamiento3,"elenaruiz@otromail.com");
        gestor.addAlojamiento2Anfitrion(alojamiento4,"lucia.mora@correo.es");
        gestor.addAlojamiento2Anfitrion(alojamiento5,"elenaruiz@otromail.com");
        System.out.println("-----------------------------------");
        System.out.println("Imprimimos la info de un alojamiento");
        System.out.println("-----------------------------------");
        alojamiento2.print();
        System.out.println("---------------------------------");
        System.out.println("Imprimimos la info de un anfitrion");
        System.out.println("---------------------------------");
        anfitrion3.print();
        System.out.println("-----------------------------------------");
        System.out.println("Eliminamos un anfitrion a partir de su email");
        System.out.println("-----------------------------------------");
        gestor.removeAnfitrion("elenaruiz@otromail.com");
        System.out.println("---------------------------------");
        System.out.println("Imprimimos la lista de anfitriones");
        System.out.println("---------------------------------");
        gestor.printAllAnfitriones();
        System.out.println("-----------------------------------------------------");
        System.out.println("Imprimimos el total de alojamientos en la plataforma");
        System.out.println("-----------------------------------------------------");
        gestor.printAllAlojamientos();
    }
}
```

[3] En el curso virtual, junto con el enunciado de la práctica, puede descargar el código fuente de esta clase LanzadorEtapa2_1.

Al ejecutar la clase anterior, el sistema deberá imprimir por pantalla exactamente lo siguiente:

```
-----------------------------------
Creamos la plataforma de alojamientos
-----------------------------------
```

```
Creada plataforma de alojamientos Vacanza
------------------
Creamos anfitriones
------------------
Creado anfitrion lucia.mora@correo.es
Creado anfitrion pablosanz@mimail.com
Creado anfitrion elenaruiz@otromail.com
-------------------
Añadimos anfitriones
-------------------
El anfitrion con email lucia.mora@correo.es ha sido añadido
El anfitrion con email pablosanz@mimail.com ha sido añadido
El anfitrion con email elenaruiz@otromail.com ha sido añadido
-------------------
Creamos alojamientos
-------------------
Creado alojamiento con id 1
Creado alojamiento con id 2
Creado alojamiento con id 3
Creado alojamiento con id 4
Creado alojamiento con id 5
-----------------------------------------
Añadimos los alojamientos a los anfitriones
-----------------------------------------
El estado del alojamiento con id 1 ha sido modificado a estado DISPONIBLE
El alojamiento con id 1 se ha añadido al anfitrion lucia.mora@correo.es
El estado del alojamiento con id 2 ha sido modificado a estado DISPONIBLE
El alojamiento con id 2 se ha añadido al anfitrion pablosanz@mimail.com
El estado del alojamiento con id 3 ha sido modificado a estado DISPONIBLE
El alojamiento con id 3 se ha añadido al anfitrion elenaruiz@otromail.com
El estado del alojamiento con id 4 ha sido modificado a estado DISPONIBLE
El alojamiento con id 4 se ha añadido al anfitrion lucia.mora@correo.es
El estado del alojamiento con id 5 ha sido modificado a estado DISPONIBLE
El alojamiento con id 5 se ha añadido al anfitrion elenaruiz@otromail.com
-----------------------------------
Imprimimos la info de un alojamiento
-----------------------------------
ID: 2
Titulo: Casa rural en la sierra de Gredos
Precio por noche: 140.0
Descripcion: Casa de piedra con chimenea y barbacoa. Se admiten mascotas.
Estado: DISPONIBLE
-----------------------------------
Imprimimos la info de un anfitrion
-----------------------------------
------------
Nombre: Elena Ruiz Cano
Email: elenaruiz@otromail.com
------------
ID: 3
Titulo: Estudio en la playa de Valencia
Precio por noche: 70.0
Descripcion: Estudio nuevo a 50 metros de la playa. Cocina equipada.
Estado: DISPONIBLE
ID: 5
Titulo: Cabaña junto al lago
Precio por noche: 85.0
Descripcion: Cabaña de madera junto al lago con barbacoa y zona de pesca.
Estado: DISPONIBLE
-----------------------------------------
Eliminamos un anfitrion a partir de su email
-----------------------------------------
El anfitrion con email elenaruiz@otromail.com ha sido eliminado
-----------------------------------
Imprimimos la lista de anfitriones
-----------------------------------
------------
Nombre: Lucía Mora Vidal
Email: lucia.mora@correo.es
------------
ID: 1
Titulo: Apartamento céntrico en Madrid
Precio por noche: 95.0
Descripcion: Luminoso, junto a la Puerta del Sol. Wifi y aire acondicionado.
Estado: DISPONIBLE
ID: 4
```

```
Titulo: Loft con vistas en Barcelona
Precio por noche: 120.5
Descripcion: Loft moderno con vistas a la ciudad, cerca del metro.
Estado: DISPONIBLE
------------
Nombre: Pablo Sanz Rey
Email: pablosanz@mimail.com
------------
ID: 2
Titulo: Casa rural en la sierra de Gredos
Precio por noche: 140.0
Descripcion: Casa de piedra con chimenea y barbacoa. Se admiten mascotas.
Estado: DISPONIBLE
--------------------------------------------------
Imprimimos el total de alojamientos en la plataforma
--------------------------------------------------
ID: 1
Titulo: Apartamento céntrico en Madrid
Precio por noche: 95.0
Descripcion: Luminoso, junto a la Puerta del Sol. Wifi y aire acondicionado.
Estado: DISPONIBLE
ID: 4
Titulo: Loft con vistas en Barcelona
Precio por noche: 120.5
Descripcion: Loft moderno con vistas a la ciudad, cerca del metro.
Estado: DISPONIBLE
ID: 2
Titulo: Casa rural en la sierra de Gredos
Precio por noche: 140.0
Descripcion: Casa de piedra con chimenea y barbacoa. Se admiten mascotas.
Estado: DISPONIBLE
```

3.1.6. Comprobación del funcionamiento de las funcionalidades más avanzadas (parte Opcional)

A continuación, vamos a probar el comportamiento del resto de funcionalidades que debe contemplar la clase `PlataformaAlojamientos`. Para ello vamos a crear una nueva clase `LanzadorEtapa2_2` que nos permita probar los métodos definidos para mostrar:

- El conjunto de alojamientos que se ofertan en la plataforma y el listado según su estado, los alojamientos disponibles u ocupados dentro del sistema.
- El conjunto de alojamientos que ofrece un anfitrión, con un determinado estado y a partir de su correo electrónico.
- Los alojamientos cuya descripción contenga una determinada palabra clave.

El código siguiente, correspondiente a la clase `LanzadorEtapa2_2`, incluye las instrucciones contenidas en el anterior lanzador, cuya salida ahora ya no será comentada.

```
public class LanzadorEtapa2_2 {
    public static void main(String[] args) {
        System.out.println("-------------------------------------");
        System.out.println("Creamos la plataforma de alojamientos");
        System.out.println("-------------------------------------");
        PlataformaAlojamientos gestor = new PlataformaAlojamientos("Vacanza");
        System.out.println("-------------------");
        System.out.println("Creamos anfitriones");
        System.out.println("-------------------");
        Anfitrion anfitrion1 = new Anfitrion("Lucía Mora Vidal","lucia.mora@correo.es");
        Anfitrion anfitrion2 = new Anfitrion("Pablo Sanz Rey","pablosanz@mimail.com");
        Anfitrion anfitrion3 = new Anfitrion("Elena Ruiz Cano","elenaruiz@otromail.com");
        System.out.println("--------------------");
        System.out.println("Añadimos anfitriones");
        System.out.println("--------------------");
        gestor.addAnfitrion(anfitrion1);
        gestor.addAnfitrion(anfitrion2);
        gestor.addAnfitrion(anfitrion3);
        System.out.println("--------------------");
        System.out.println("Creamos alojamientos");
        System.out.println("-------------------");
        Alojamiento alojamiento1 = new Alojamiento(1,"Apartamento céntrico en Madrid",95,
                "Luminoso, junto a la Puerta del Sol. Wifi y aire acondicionado.",T_Estado.DISPONIBLE);
```

```
        Alojamiento alojamiento2 = new Alojamiento(2,"Casa rural en la sierra de Gredos",140,
                "Casa de piedra con chimenea y barbacoa. Se admiten mascotas.",T_Estado.DISPONIBLE);
        Alojamiento alojamiento3 = new Alojamiento(3,"Estudio en la playa de Valencia",70,
                "Estudio nuevo a 50 metros de la playa. Cocina equipada.",T_Estado.DISPONIBLE);
        Alojamiento alojamiento4 = new Alojamiento(4,"Loft con vistas en Barcelona",120.5,
                "Loft moderno con vistas a la ciudad, cerca del metro.",T_Estado.DISPONIBLE);
        Alojamiento alojamiento5 = new Alojamiento(5,"Cabaña junto al lago",85,
                "Cabaña de madera junto al lago con barbacoa y zona de pesca.",T_Estado.DISPONIBLE);
        System.out.println("-----------------------------------------");
        System.out.println("Añadimos los alojamientos a los anfitriones");
        System.out.println("-----------------------------------------");
        gestor.addAlojamiento2Anfitrion(alojamiento1,"lucia.mora@correo.es");
        gestor.addAlojamiento2Anfitrion(alojamiento2,"pablosanz@mimail.com");
        gestor.addAlojamiento2Anfitrion(alojamiento3,"elenaruiz@otromail.com");
        gestor.addAlojamiento2Anfitrion(alojamiento4,"lucia.mora@correo.es");
        gestor.addAlojamiento2Anfitrion(alojamiento5,"elenaruiz@otromail.com");
        System.out.println("-----------------------------------");
        System.out.println("Imprimimos la info de un alojamiento");
        System.out.println("-----------------------------------");
        alojamiento2.print();
        System.out.println("---------------------------------");
        System.out.println("Imprimimos la info de un anfitrion");
        System.out.println("---------------------------------");
        anfitrion3.print();
        System.out.println("-----------------------------------------");
        System.out.println("Eliminamos un anfitrion a partir de su email");
        System.out.println("-----------------------------------------");
        gestor.removeAnfitrion("elenaruiz@otromail.com");
        System.out.println("---------------------------------");
        System.out.println("Imprimimos la lista de anfitriones");
        System.out.println("---------------------------------");
        gestor.printAllAnfitriones();
        System.out.println("-----------------------------------------------------");
        System.out.println("Imprimimos el total de alojamientos en la plataforma");
        System.out.println("-----------------------------------------------------");
        gestor.printAllAlojamientos();
        System.out.println("--------------------------------------");
        System.out.println("Eliminamos un alojamiento a un anfitrion");
        System.out.println("--------------------------------------");
        gestor.removeAlojamiento2Anfitrion(1,anfitrion1.getEmail());
        gestor.printAlojamientosByAnfitrion(anfitrion1.getEmail());
        System.out.println("---------------------------------");
        System.out.println("Imprimimos todos los alojamientos");
        System.out.println("---------------------------------");
        gestor.printAllAlojamientos();
        // **************
        // PARTE OPCIONAL
        // **************
        alojamiento2.setEstado(T_Estado.OCUPADO);
        System.out.println("--------------------------------------");
        System.out.println("Imprimimos los alojamientos disponibles");
        System.out.println("--------------------------------------");
        gestor.printAlojamientosByState(T_Estado.DISPONIBLE);
        System.out.println("-----------------------------------");
        System.out.println("Imprimimos los alojamientos ocupados");
        System.out.println("-----------------------------------");
        gestor.printAlojamientosByState(T_Estado.OCUPADO);
        System.out.println("-----------------------------------------------------");
        System.out.println("Imprimimos los alojamientos disponibles de un anfitrion");
        System.out.println("-----------------------------------------------------");
        gestor.printAlojamientosByAnfitrion(anfitrion1.getEmail(),T_Estado.DISPONIBLE);
        System.out.println("-------------------------------------------------------------");
        System.out.println("Imprimimos los alojamientos que contienen la palabra \"barbacoa\"");
        System.out.println("-------------------------------------------------------------");
        gestor.searchAlojamientosByKeyword("barbacoa");
    }
}
```

Al ejecutar la clase (y continuando a partir de la última instrucción contenida en el `LanzadorEtapa2_1`) se obtiene la salida siguiente:

```
--------------------------------------
Eliminamos un alojamiento a un anfitrion
--------------------------------------
El alojamiento con id 1 se ha eliminado de la lista del anfitrion lucia.mora@correo.es
```

```
ID: 4
Titulo: Loft con vistas en Barcelona
Precio por noche: 120.5
Descripcion: Loft moderno con vistas a la ciudad, cerca del metro.
Estado: DISPONIBLE
--------------------------------
Imprimimos todos los alojamientos
--------------------------------
ID: 4
Titulo: Loft con vistas en Barcelona
Precio por noche: 120.5
Descripcion: Loft moderno con vistas a la ciudad, cerca del metro.
Estado: DISPONIBLE
ID: 2
Titulo: Casa rural en la sierra de Gredos
Precio por noche: 140.0
Descripcion: Casa de piedra con chimenea y barbacoa. Se admiten mascotas.
Estado: DISPONIBLE
El estado del alojamiento con id 2 ha sido modificado a estado OCUPADO
-------------------------------------
Imprimimos los alojamientos disponibles
-------------------------------------
ID: 4
Titulo: Loft con vistas en Barcelona
Precio por noche: 120.5
Descripcion: Loft moderno con vistas a la ciudad, cerca del metro.
Estado: DISPONIBLE
-----------------------------------
Imprimimos los alojamientos ocupados
-----------------------------------
ID: 2
Titulo: Casa rural en la sierra de Gredos
Precio por noche: 140.0
Descripcion: Casa de piedra con chimenea y barbacoa. Se admiten mascotas.
Estado: OCUPADO
-----------------------------------------------------
Imprimimos los alojamientos disponibles de un anfitrion
-----------------------------------------------------
ID: 4
Titulo: Loft con vistas en Barcelona
Precio por noche: 120.5
Descripcion: Loft moderno con vistas a la ciudad, cerca del metro.
Estado: DISPONIBLE
-------------------------------------------------------------
Imprimimos los alojamientos que contienen la palabra "barbacoa"
-------------------------------------------------------------
ID: 2
Titulo: Casa rural en la sierra de Gredos
Precio por noche: 140.0
Descripcion: Casa de piedra con chimenea y barbacoa. Se admiten mascotas.
Estado: OCUPADO
```

Es importante probar el código con otros datos, modificando el valor de las diferentes variables usadas para chequear el funcionamiento de los diferentes métodos y comprobar así el correcto funcionamiento del sistema.

ETAPA 3

4.1. ETAPA 3.1: Uso de la herencia y polimorfismo

4.1.1. Objetivos

En esta parte de la práctica vamos a extender el sistema para que nos permita gestionar alojamientos de dos categorías diferentes: urbanos y rurales. Cada una de ellas incluirá nuevos atributos y variaciones en sus métodos. Para este tipo de situaciones, los lenguajes basados en el paradigma de la programación orientada a objetos, como Java, disponen de un mecanismo llamado Herencia. La herencia permite construir una nueva clase que "herede" las características de una ya existente. Por tanto, esta clase "heredera" o "hija" tendrá toda la funcionalidad de la clase de la que hereda (clase padre, siguiendo con la analogía), permitiendo además la inclusión de nuevos métodos y campos, así como la sobreescritura de los métodos ya existentes (si fuera necesario), para extender sus funcionalidades. De este modo, la clase resultante será una extensión de la clase que ya teníamos.

Por otro lado, cuando el sistema gestiona alojamientos de la categoría urbano y rural, la clase `Anfitrion`, con la lista de alojamientos que pone en oferta un anfitrión, no necesitará saber si cada uno de los alojamientos ofertados es de tipo urbano o rural. En este modo, en las listas de objetos usamos un mecanismo llamado Polimorfismo, que permite que el sistema siga funcionando sin modificar el código que ya teníamos. Es decir, las llamadas a métodos de la clase `Anfitrion` que involucran alojamientos seguirán siendo válidas sin necesidad de modificar nada, y en este caso añadirán a la lista u obtendrán de ella el conjunto de alojamientos ofertados por un anfitrión, independientemente de que la categoría del alojamiento sea urbana o rural.

En esta etapa 3 estudiaremos el uso y posibilidades que nos ofrecen este tipo de mecanismos de la programación orientada a objetos.

4.1.2. Requisitos previos

Para la realización de esta etapa es necesario dominar los conceptos de objeto, clase y la relación entre clases denominada composición, estudiados hasta este punto.

4.1.3. Añadiendo alojamientos de diferentes tipos

A diferencia de un alojamiento de tipo rural, que se caracteriza por tener una superficie de parcela y disponer o no de piscina, los alojamientos de la categoría urbana se caracterizarán dentro de nuestro sistema por tener un número de habitaciones, una superficie en metros cuadrados y una planta. El conjunto de atributos que definían hasta ahora un alojamiento (id, título, precio por noche, descripción y estado) son compartidos por ambas categorías. En esta tercera etapa de la práctica se deben diseñar e implementar las clases necesarias para cubrir estas especificaciones:

Clase	Atributos propios (en este orden en el constructor, tras los de Alojamiento)
AlojamientoUrbano	int numHabitaciones, double superficie, int planta
AlojamientoRural	double superficieParcela, boolean piscina

Cada clase hija deberá incluir los métodos de consulta y modificación de sus atributos y sobreescribir el método `print()` para mostrar, a continuación de los datos comunes, la información propia con el formato que se muestra en las salidas de este apartado (en los alojamientos rurales, la existencia de piscina se muestra como "Si" o "No").

Pero ahora hay que considerar un aspecto importante. Dado que ya no tendremos alojamientos "en general", sino que serán de una categoría u otra, ya no tiene sentido que podamos instanciar objetos de la clase `Alojamiento`, sino que habrá que hacerlo de una clase `AlojamientoUrbano` o

`AlojamientoRural`. Para ello, es necesario modificar la clase `Alojamiento` que ya teníamos para convertirla en una **clase abstracta**, lo que significa que ya no se podrán instanciar objetos de la clase `Alojamiento`. La finalidad de una clase abstracta es servir como clase base para otras clases que hereden de ella, ya que una clase abstracta recoge las características y funcionalidades comunes de todas sus clases hijas. Deberá considerarse con atención la visibilidad de los atributos de la clase `Alojamiento` ahora que es una clase abstracta.

Como en las etapas previas, vamos a crear una nueva clase llamada `LanzadorEtapa3_1`, que contenga un método `main()`, cuyo cuerpo contenga el código fuente que se muestra a continuación[4]:

```
public class LanzadorEtapa3_1 {
    public static void main(String[] args) {
        System.out.println("-------------------------------------");
        System.out.println("Creamos la plataforma de alojamientos");
        System.out.println("-------------------------------------");
        PlataformaAlojamientos gestor = new PlataformaAlojamientos("Vacanza");
        System.out.println("-------------------");
        System.out.println("Creamos anfitriones");
        System.out.println("-------------------");
        Anfitrion anfitrion1 = new Anfitrion("Lucía Mora Vidal","lucia.mora@correo.es");
        Anfitrion anfitrion2 = new Anfitrion("Pablo Sanz Rey","pablosanz@mimail.com");
        System.out.println("--------------------");
        System.out.println("Añadimos anfitriones");
        System.out.println("--------------------");
        gestor.addAnfitrion(anfitrion1);
        gestor.addAnfitrion(anfitrion2);
        System.out.println("--------------------");
        System.out.println("Creamos alojamientos");
        System.out.println("--------------------");
        Alojamiento alojamiento1 = new AlojamientoUrbano(1,"Apartamento céntrico en Madrid",95,
                "Luminoso, junto a la Puerta del Sol. Wifi y aire acondicionado.",
                T_Estado.DISPONIBLE,2,60.5,3);
        Alojamiento alojamiento2 = new AlojamientoRural(2,"Casa rural en la sierra de Gredos",140,
                "Casa de piedra con chimenea y barbacoa. Se admiten mascotas.",
                T_Estado.DISPONIBLE,1200,true);
        Alojamiento alojamiento3 = new AlojamientoRural(3,"Cabaña junto al lago",85,
                "Cabaña de madera junto al lago con zona de pesca.",
                T_Estado.DISPONIBLE,800,false);
        Alojamiento alojamiento4 = new AlojamientoUrbano(4,"Loft con vistas en Barcelona",120.5,
                "Loft moderno con vistas a la ciudad, cerca del metro.",
                T_Estado.DISPONIBLE,1,45,5);
        System.out.println("-----------------------------------------");
        System.out.println("Añadimos los alojamientos a los anfitriones");
        System.out.println("-----------------------------------------");
        gestor.addAlojamiento2Anfitrion(alojamiento1,"lucia.mora@correo.es");
        gestor.addAlojamiento2Anfitrion(alojamiento2,"pablosanz@mimail.com");
        gestor.addAlojamiento2Anfitrion(alojamiento3,"pablosanz@mimail.com");
        gestor.addAlojamiento2Anfitrion(alojamiento4,"lucia.mora@correo.es");
        System.out.println("--------------------------------------");
        System.out.println("Eliminamos un alojamiento a un anfitrion");
        System.out.println("--------------------------------------");
        gestor.removeAlojamiento2Anfitrion(1,anfitrion1.getEmail());
        gestor.printAlojamientosByAnfitrion(anfitrion1.getEmail());
        System.out.println("--------------------------------");
        System.out.println("Imprimimos todos los alojamientos");
        System.out.println("--------------------------------");
        gestor.printAllAlojamientos();
        // **************
        // PARTE OPCIONAL
        // **************
        alojamiento2.setEstado(T_Estado.OCUPADO);
        System.out.println("--------------------------------------");
        System.out.println("Imprimimos los alojamientos disponibles");
        System.out.println("--------------------------------------");
        gestor.printAlojamientosByState(T_Estado.DISPONIBLE);
        System.out.println("----------------------------------");
        System.out.println("Imprimimos los alojamientos ocupados");
        System.out.println("----------------------------------");
        gestor.printAlojamientosByState(T_Estado.OCUPADO);
        System.out.println("-------------------------------------------------------");
        System.out.println("Imprimimos los alojamientos disponibles de un anfitrion");
        System.out.println("-------------------------------------------------------");
```

```
        gestor.printAlojamientosByAnfitrion(anfitrion1.getEmail(),T_Estado.DISPONIBLE);
    }
}
```

[4] En el curso virtual, junto con el enunciado de la práctica, puede descargar el código fuente de esta clase LanzadorEtapa3_1.

Al ejecutar la clase anterior, se crea una plataforma, dos anfitriones y cuatro alojamientos correspondientes a las dos categorías que estamos considerando. A continuación, haciendo uso del polimorfismo, se incluyen los alojamientos en la lista de alojamientos de los dos anfitriones, independientemente de su categoría. Por otro lado, al incluir cada alojamiento en las listas se fija por defecto su estado como disponible. El sistema deberá imprimir exactamente lo siguiente (obsérvese cómo el mecanismo del polimorfismo nos permite acceder a los alojamientos independientemente de la categoría):

```
-----------------------------------
Creamos la plataforma de alojamientos
-----------------------------------
Creada plataforma de alojamientos Vacanza
------------------
Creamos anfitriones
------------------
Creado anfitrion lucia.mora@correo.es
Creado anfitrion pablosanz@mimail.com
-------------------
Añadimos anfitriones
-------------------
El anfitrion con email lucia.mora@correo.es ha sido añadido
El anfitrion con email pablosanz@mimail.com ha sido añadido
-------------------
Creamos alojamientos
-------------------
Creado alojamiento con id 1
Creado alojamiento con id 2
Creado alojamiento con id 3
Creado alojamiento con id 4
----------------------------------------
Añadimos los alojamientos a los anfitriones
----------------------------------------
El estado del alojamiento con id 1 ha sido modificado a estado DISPONIBLE
El alojamiento con id 1 se ha añadido al anfitrion lucia.mora@correo.es
El estado del alojamiento con id 2 ha sido modificado a estado DISPONIBLE
El alojamiento con id 2 se ha añadido al anfitrion pablosanz@mimail.com
El estado del alojamiento con id 3 ha sido modificado a estado DISPONIBLE
El alojamiento con id 3 se ha añadido al anfitrion pablosanz@mimail.com
El estado del alojamiento con id 4 ha sido modificado a estado DISPONIBLE
El alojamiento con id 4 se ha añadido al anfitrion lucia.mora@correo.es
---------------------------------------
Eliminamos un alojamiento a un anfitrion
---------------------------------------
El alojamiento con id 1 se ha eliminado de la lista del anfitrion lucia.mora@correo.es
ID: 4
Titulo: Loft con vistas en Barcelona
Precio por noche: 120.5
Descripcion: Loft moderno con vistas a la ciudad, cerca del metro.
Estado: DISPONIBLE
Habitaciones: 1
Superficie: 45.0 m2
Planta: 5
--------------------------------
Imprimimos todos los alojamientos
--------------------------------
ID: 4
Titulo: Loft con vistas en Barcelona
Precio por noche: 120.5
Descripcion: Loft moderno con vistas a la ciudad, cerca del metro.
Estado: DISPONIBLE
Habitaciones: 1
Superficie: 45.0 m2
Planta: 5
ID: 2
Titulo: Casa rural en la sierra de Gredos
Precio por noche: 140.0
Descripcion: Casa de piedra con chimenea y barbacoa. Se admiten mascotas.
Estado: DISPONIBLE
```

```
Superficie de parcela: 1200.0 m2
Piscina: Si
ID: 3
Titulo: Cabaña junto al lago
Precio por noche: 85.0
Descripcion: Cabaña de madera junto al lago con zona de pesca.
Estado: DISPONIBLE
Superficie de parcela: 800.0 m2
Piscina: No
El estado del alojamiento con id 2 ha sido modificado a estado OCUPADO
--------------------------------------
Imprimimos los alojamientos disponibles
--------------------------------------
ID: 4
Titulo: Loft con vistas en Barcelona
Precio por noche: 120.5
Descripcion: Loft moderno con vistas a la ciudad, cerca del metro.
Estado: DISPONIBLE
Habitaciones: 1
Superficie: 45.0 m2
Planta: 5
ID: 3
Titulo: Cabaña junto al lago
Precio por noche: 85.0
Descripcion: Cabaña de madera junto al lago con zona de pesca.
Estado: DISPONIBLE
Superficie de parcela: 800.0 m2
Piscina: No
----------------------------------
Imprimimos los alojamientos ocupados
----------------------------------
ID: 2
Titulo: Casa rural en la sierra de Gredos
Precio por noche: 140.0
Descripcion: Casa de piedra con chimenea y barbacoa. Se admiten mascotas.
Estado: OCUPADO
Superficie de parcela: 1200.0 m2
Piscina: Si
------------------------------------------------------
Imprimimos los alojamientos disponibles de un anfitrion
------------------------------------------------------
ID: 4
Titulo: Loft con vistas en Barcelona
Precio por noche: 120.5
Descripcion: Loft moderno con vistas a la ciudad, cerca del metro.
Estado: DISPONIBLE
Habitaciones: 1
Superficie: 45.0 m2
Planta: 5
```

Al igual que en las etapas anteriores, es importante probar el código con diferentes datos y comprobar así el correcto funcionamiento del sistema.

FECHAS Y NORMAS DE ENTREGA

5.1. Sesiones presenciales

La realización de la práctica se llevará a cabo en los Centros Asociados, siendo las sesiones organizadas y supervisadas por el tutor de la asignatura. Habrá como mínimo tres sesiones presenciales de obligatoria asistencia. Los estudiantes contactarán con su centro asociado para informarse sobre cuándo tendrán que asistir a las sesiones. A lo largo de las tres sesiones se abordarán las diferentes partes de las que consta la práctica. Las fechas orientativas para la realización de cada una de las etapas serán:

- Finales de marzo. Realización de la primera sesión presencial (Etapa 1).
- Mediados de abril. Realización de la segunda sesión presencial (Etapa 2).
- Principios de mayo. Realización de la tercera sesión presencial (Etapa 3).

5.2. Normas de entrega

La práctica se entregará a través de una tarea definida en el entorno virtual de la asignatura. Cada estudiante creará y comprimirá una carpeta nombrada con su DNI y primer apellido separados por un guion (“12345678X-gonzalez”). Esta carpeta contendrá:

- Una memoria de no más de 6 páginas donde se expliquen las decisiones tomadas, así como el diseño (ver sección 1.2 de este documento) realizado en cada parte de la práctica, incluyendo el diagrama de clases de la última etapa.
- Los ficheros .java, sin caracteres especiales; por ejemplo “ñ” o tildes finales, es decir, únicamente los de la última etapa.

NOTA IMPORTANTE: Los nombres de los ficheros y carpetas/paquetes que compongan la práctica entregada deben contener SOLO caracteres correspondientes a las letras de la A a la Z, tanto mayúsculas como minúsculas, números del 0 al 9 y los caracteres especiales ‘-’ y ‘_’. No deben utilizarse otros, tales como tildes o símbolos.

5.3. Fechas de entrega

Para la convocatoria ordinaria (junio) todos los estudiantes deberán entregar su práctica en el entorno virtual antes del **[fecha de entrega]**. Los tutores deberán evaluar las prácticas y subir las notas al entorno virtual antes del **[fecha de notas]**. Las fechas correspondientes a la convocatoria extraordinaria (septiembre) se publicarán más adelante en el curso virtual.

NOTA IMPORTANTE: Los tutores deben cumplir unos requisitos ante los estudiantes porque la práctica cuenta para la calificación de la asignatura. Por tanto, antes de entregar las calificaciones definitivas al equipo docente deberán: (1) informar de la nota de las prácticas a los estudiantes; (2) establecer un día de revisión de notas de prácticas previo a la fecha tope de entrega de las calificaciones al equipo docente.

Es importante que se mantengan todos los identificadores definidos en el enunciado, es decir, el nombre de las clases, atributos y métodos deben ser tal y como se definen en este enunciado.

5.4. Evaluación de la práctica

Las prácticas tienen carácter **INDIVIDUAL**; para evitar posibles copias todas las prácticas pasarán por un software detector de ejemplares similares. La detección de prácticas copiadas implicará un **SUSPENSO** en TODO el curso, es decir, convocatorias de junio y septiembre, para todos los implicados.

Es requisito indispensable para aprobar el examen la asistencia a las 3 sesiones obligatorias de la práctica, así como superar la propia práctica. Las prácticas NO se guardan de un curso para otro.

5.5. Preguntas al equipo docente

Las preguntas sobre la instalación del entorno de desarrollo, puesta en funcionamiento y errores de compilación deben remitirse a los tutores de los centros asociados. El equipo docente atenderá preguntas de carácter metodológico y de diseño, preferentemente a través de los foros del curso virtual de la asignatura y del siguiente correo electrónico: **[correo del equipo docente]**. Los datos de contacto de cada uno de los miembros del equipo docente son:

Nombre y cargo	Tutorías	Teléfono	Correo
[Coordinador/a de la asignatura]	[días y horas]	[teléfono]	[correo]
[Profesor/a 2]	[días y horas]	[teléfono]	[correo]
[Profesor/a 3]	[días y horas]	[teléfono]	[correo]